

Язык программирования Python

В 1991 году Гвидо ван Россумом был создан язык программирования Python. Назван он был в честь любимого шоу создателя— «Летающий цирк Монти Пайтона» («Monty Python's Flying Circus»). Существует 3 версии Python. В настоящее время самым актуальным является Python 3.

Python – это интерпретируемый, универсальный и передовой язык программирования, который используется в следующих областях:

- анализ данных;
- написание прикладных программных систем;
- разработка графических интерфейсов;
- написание сетевого программного обеспечения;
- взаимодействие с базами данных.

Принципы языка Python, они же философия, к которой призывают стремиться «носители» языка, сформулированы в The Zen of Python:

- Красивое лучше, чем уродливое.
- Явное лучше, чем неявное.
- Простое лучше, чем сложное.
- Сложное лучше, чем запутанное.
- Плоское лучше, чем вложенное.
- Разреженное лучше, чем плотное.
- Читаемость имеет значение.
- Особые случаи не настолько особые, чтобы нарушать правила.
- При этом практичность важнее безупречности.
- Ошибки никогда не должны замалчиваться.
- Если они не замалчиваются явно.
- Встретив двусмысленность, отбрось искушение угадать.
- Должен существовать один и желательно только один очевидный способ сделать это.
- Хотя он поначалу может быть и не очевиден, если вы не голландец.
- Сейчас лучше, чем никогда.

- Хотя никогда зачастую лучше, чем прямо сейчас.
- Если реализацию сложно объяснить – идея плоха.
- Если реализацию легко объяснить – идея, возможно, хороша.
- Пространства имен – отличная штука! Будем делать их больше!

Данная философия подходит к разработке на любом языке, и нередко разработчики на других языках ссылаются на The Zen of Python.

К особенностям Python относится *динамическая типизация*. Типы объектов во время выполнения программы строго не определены, они могут изменяться. Так же для определения переменных не нужно объявлять их тип.

Для выделения блоков кода используется «:».

Python позволяет писать операции на одной строке с помощью разделителя «;», но делать так не рекомендуется, такой код достаточно сложно читать.

Однострочный комментарий в Python начинается с #, а многострочный выделяется с помощью '''...''' или """...""".

Python поддерживает функциональное программирование.

Существуют два режима запуска Python:

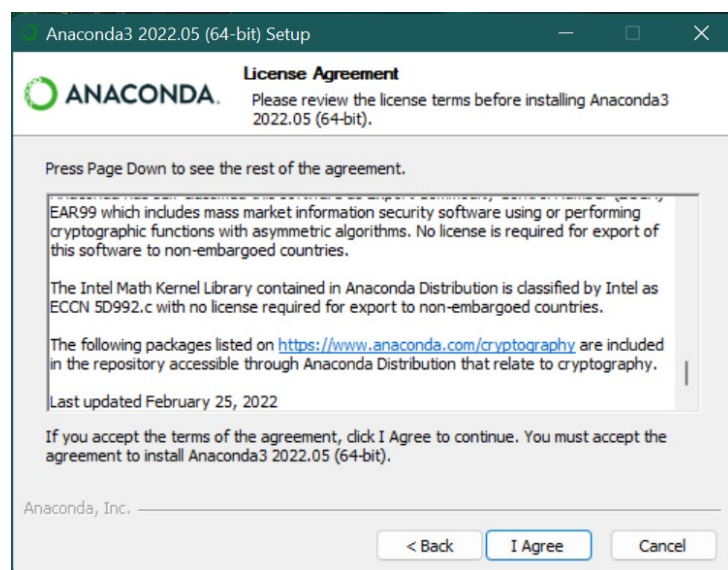
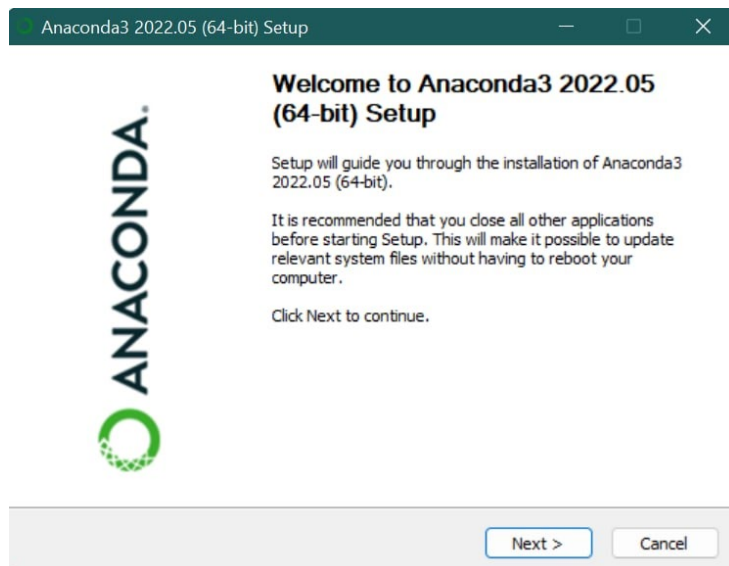
1. Стандартный режим. Необходимо сохранить файл с кодом с расширением .py, затем подать этот файл на вход интерпретатору для запуска кода.
2. Интерактивный режим. Результат работы команд виден сразу.

Установка Python

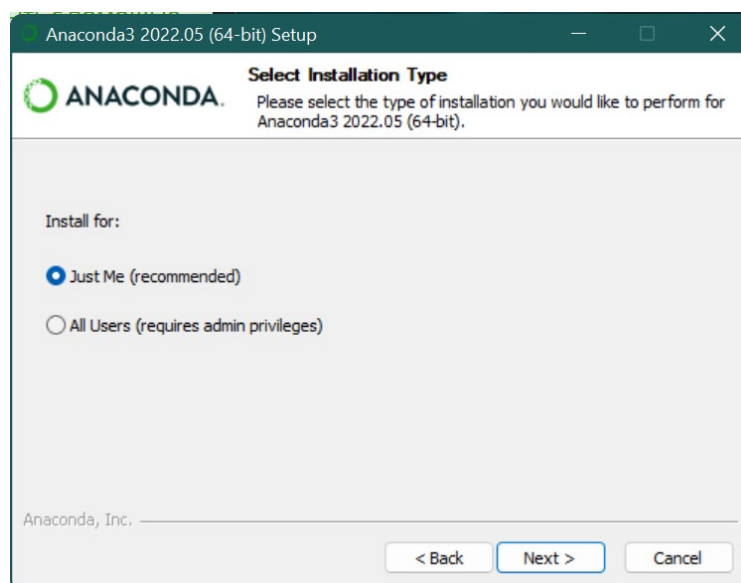
Один из самых простых способов установки Python – через дистрибутив Anaconda. Он содержит не только Python, но и необходимые библиотеки.

Скачать Anaconda можно тут: <https://www.anaconda.com/>

Далее необходимо установить Anaconda.

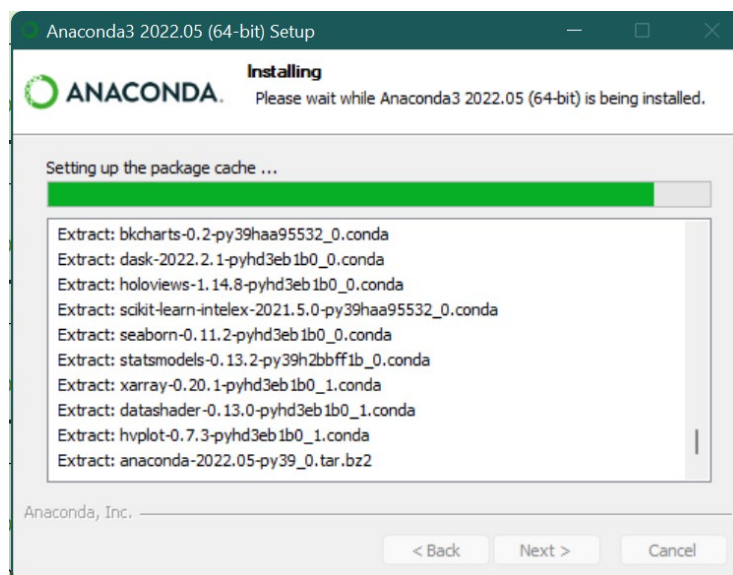
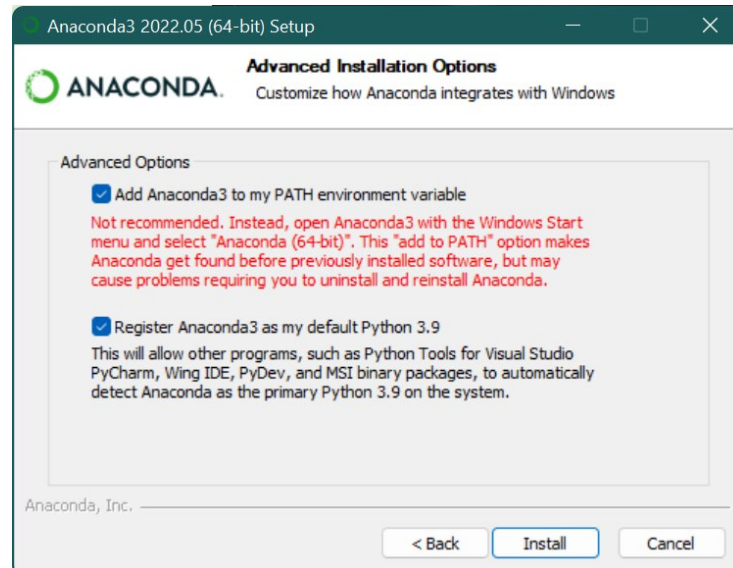


Необходимо ознакомиться с лицензией и нажать I agree.

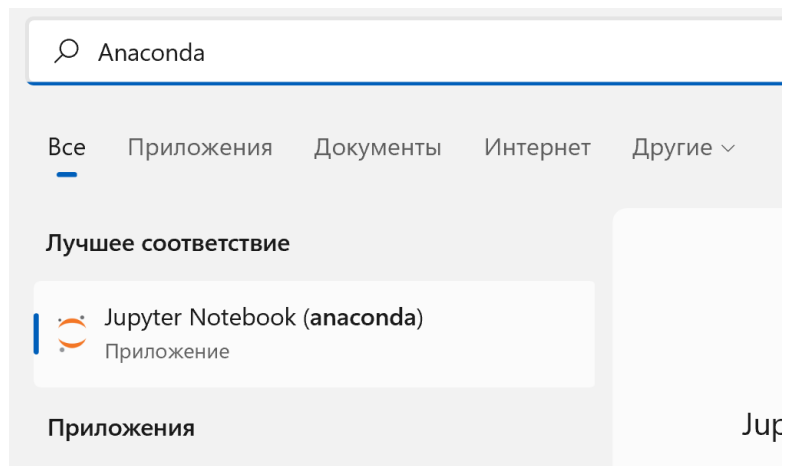


Далее предлагается внести изменения в окружение для того, чтобы Python мог по умолчанию запускаться из Anaconda. Выбрана первая галочка

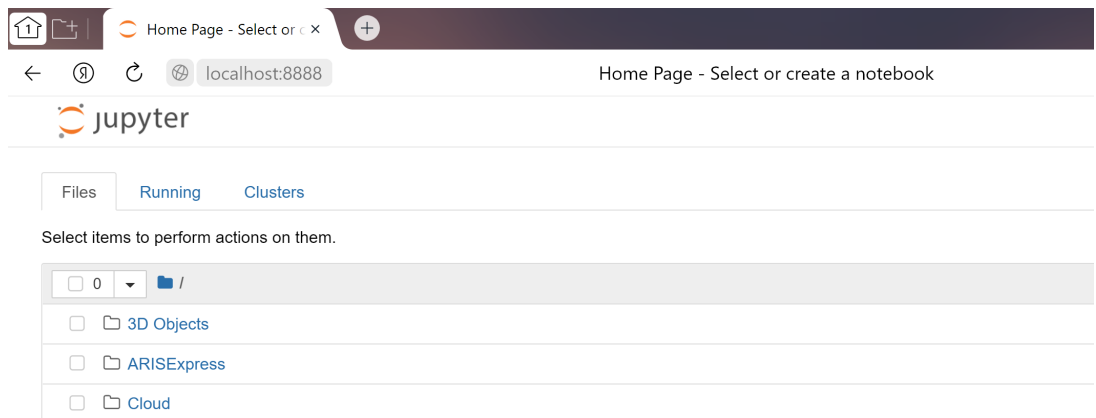
для того, чтобы в пункте PATH был прописан путь до расположения Anaconda. В дальнейшем это позволит писать команды в консоли Windows (cmd) команды для установки/удаления/обновления библиотек Python, например `pip install/conda install`



Проверим, работает ли Jupyter Notebook.

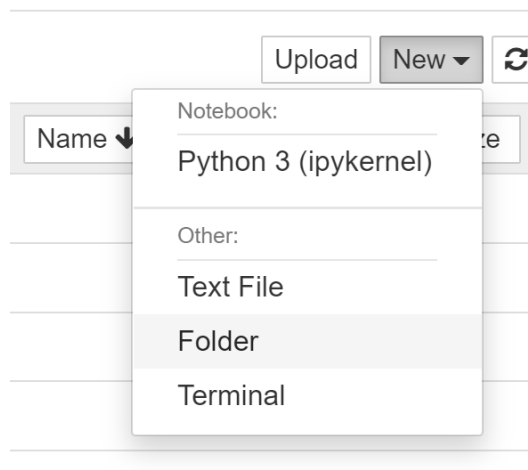


Должна открыться новая вкладка в браузере.

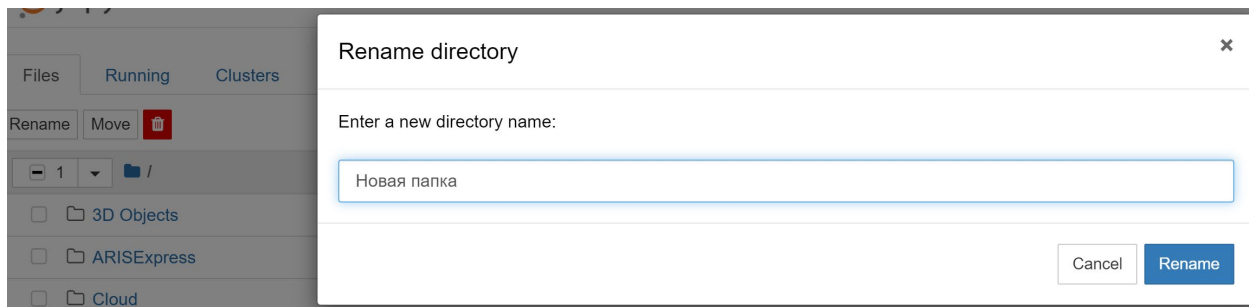


Работа с Jupyter Notebook

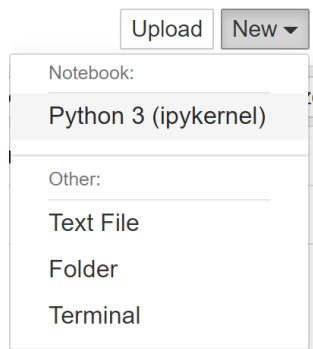
Создадим новую папку.



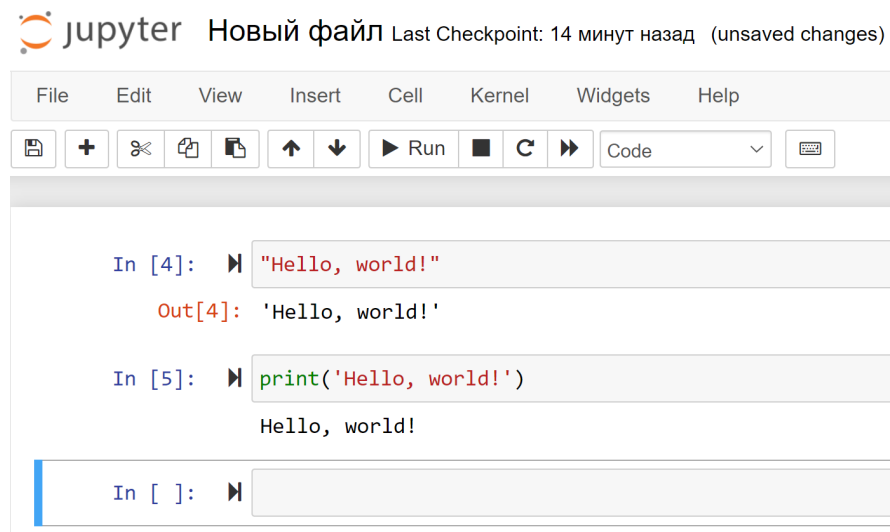
Для того, чтобы переименовать папку необходимо выбрать ее и в левом верхнем углу выбрать «Rename».



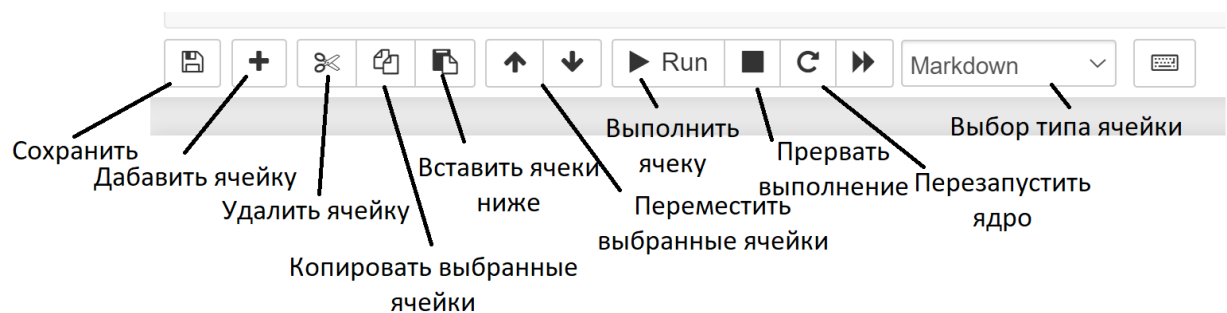
Создадим новый файл IPython. Файл с расширением `ipynb`.



Выведем предложение «Hello, world!»

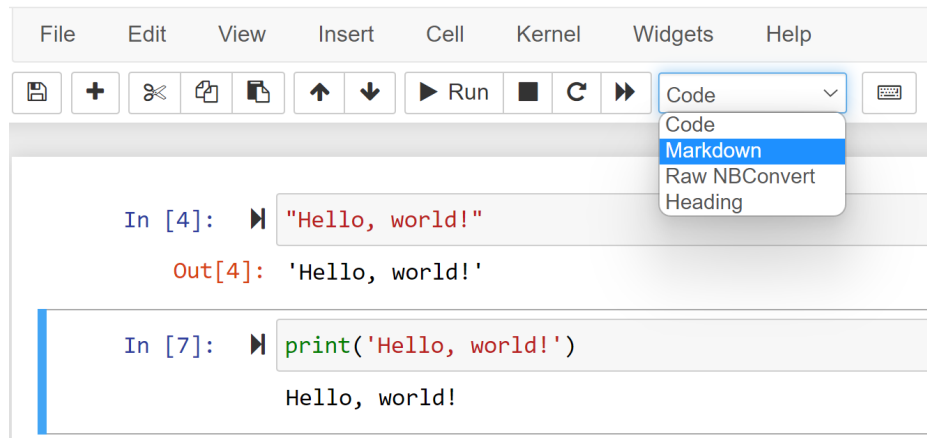


Для запуска кода в ячейке необходимо использовать комбинацию клавиш «Shift+Enter».



Для того, чтобы выделить несколько ячеек, необходимо с зажатым «Shift» их выбрать.

Существует два вида ячеек в Jupyter Notebook: ячейка для выполнения кода (Code) и ячейка для написания текста (Markdown). Тип ячейки всегда можно изменить, например:



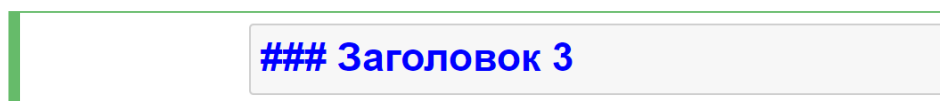
Результат:



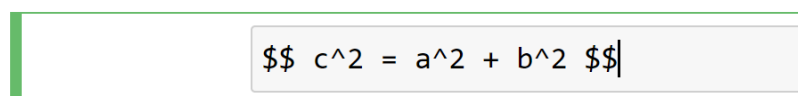
Для того, чтобы сделать заголовок в текстовой ячейке, необходимо использовать #. Чем больше #, тем меньше будет уровень заголовка. Пример:

Заголовок 1

Заголовок 2



В ячейках для текста также можно вставлять формулы с помощью знака \$\$, например:



$$c^2 = a^2 + b^2$$

Переменные в Python

Переменная — именованная типизированная ячейка памяти.

Список имен, которыми нельзя называть переменные, эти названия зарезервированы в Python:

- and
- assert
- break
- class
- continue
- def
- del
- except
- False
- for
- from
- global
- if / elif / else
- import
- in
- is
- lambda
- list
- None
- nonlocal
- not
- or
- pass
- raise
- return
- True

- try / finally
- while
- with / as
- yield

Типы переменных Python

В Python 3 есть следующие базовые типы переменных: bool, str, float, int, complex.

Целочисленные переменные (int) и основные операции с ними:

```
In [2]: a = 5

In [3]: print(type(a))
<class 'int'>

In [4]: b = 2

In [5]: A = a + b
        B = a * b
        C = a / b
        D = a // b # деление с округлением до целой части
        E = a ** b # a в степени b

In [6]: print(A,B,C,D,E)
7 10 2.5 2 25

In [7]: -a / b
Out[7]: -2.5

In [8]: -a//b
Out[8]: -3

In [9]: -(a//b)
Out[9]: -2
```

При целочисленном делении отрицательного целого числа -5 на 2 получаем -3, идет округление в меньшую сторону. Чтобы получить такой же результат, как и в случае с положительными числами, необходимо сначала поделить положительные числа, а затем добавить минус (In [9]).

Метод input() позволяет ввести данные от пользователя, пример:

```
1 a = input()
2 b = int(input())
```

```
1
2
```

```
1 print(a, type(a))
2 print(b, type(b))
```

```
1 <class 'str'>
2 <class 'int'>
```

Если нет явного указания типа данных ввода, то по умолчанию – это str.

Комплексные переменные (complex):

```
In [10]: 1 c = 5 + 9j
```

```
In [11]: 1 print(type(c))
```

```
<class 'complex'>
```

```
In [12]: 1 x = complex(3,2)
2 print(x)
```

```
(3+2j)
```

```
In [13]: 1 print(c+x)
2 print(c-x)
3 print(c*x)
4 print(c/x)
```

```
(8+11j)
(2+7j)
(-3+37j)
(2.5384615384615388+1.307692307692308j)
```

```
In [14]: 1 print(x.conjugate()) #сопряженное число
```

```
(3-2j)
```

```
In [15]: 1 print(x.imag) #мнимая часть
2 print(x.real) #действительная часть
```

```
2.0
3.0
```

Вещественные числа (float):

```
In [16]: ▶ a = 5.0  
b = 3.2  
print(type(b))  
  
<class 'float'>
```

```
In [17]: ▶ print(a+b)  
print(a-b)  
print(a*b)  
print(a/b)  
print(a//b)  
  
8.2  
1.7999999999999998  
16.0  
1.5625  
1.0
```

```
In [18]: ▶ d = int(a)  
print(d)  
  
5
```

Логический тип (bool):

Логический тип (True или False) можно привести к целочисленному типу.

```
In [19]: ► a = True
          b = False
          print(type(a))

          <class 'bool'>
```

```
In [20]: ► print(int(a))
          print(int(b))

          1
          0
```

```
In [21]: ► print(a+a)
          print(a+b)
          print(a-b)
          print(b-a)

          2
          1
          1
          -1
```

```
In [22]: ► #операции с логическим типом
          print(True and False)
          print(True and True)
          print(False and False)

          print(True or False)
          print(True or True)
          print(False or False)
```

```
In [22]: ► #операции с логическим типом
          print(True and False)
          print(True and True)
          print(False and False)

          print(True or False)
          print(True or True)
          print(False or False)

          False
          True
          False
          True
          True
          False
```

Проверка числа, подающегося на вход:

```
In [31]: ▶ inp = int(input('Введите целое число: '))
print( inp <= 10 and inp >= 3)
```

```
Введите целое число: 5
True
```

```
In [32]: ▶ inp = int(input('Введите целое число: '))
print( inp <= 10 and inp >= 3)
```

```
Введите целое число: 11
False
```

Пустая строка определяется нулевой длиной, а при преобразовании к логическому типу (bool) пустая строка превращается в False. Если строка имеет длину, отличную от нуля, то при преобразовании к логическому типу она превращается в True.

Строковый тип (str):

В Python строка – это последовательность символов, фиксация которой начинается с 0. Строки являются неизменяемыми.

```
In [23]: ▶ a = "Hello"
b = 'Python'
print(type(a))

<class 'str'>
```

```
In [24]: ▶ print(a + b)

HelloPython
```

```
In [25]: ▶ print(a*3)

HelloHelloHello
```

```
In [26]: ▶ print(a.upper()) # верхний регистр
print(b.lower()) # нижний регистр

HELLO
python
```

```
In [27]: ▶ print(a[2])
print(a[1:3])
print(a[:3]+'000'+a[3:]) #срез

l
el
Hel000lo
```

```
In [28]: ▶ print(len(a)) #длина строки
```

```
In [29]: ► print(bool(a)) # нужна строка или нет  
print(bool(''))
```

```
True  
False
```

```
In [30]: ► help(str)
```

```
Return a formatted version of the string as described by format.  
  
__ge__(self, value, /)  
Return self>=value.  
  
__getattr__(self, name, /)  
Return getattr(self, name).  
  
__getitem__(self, key, /)  
Return self[key].  
  
__getnewargs__(...)  
  
__gt__(self, value, /)  
Return self>value.  
  
__hash__(self, /)  
Return hash(self).  
  
__iter__(self, /)
```

```
In [31]: ► a.__sizeof__() #строка в памяти занимает 54 байта
```

```
Out[31]: 54
```

`\n` – символ перевода на новую строку.

```
► print("Светает. Радуетя солнце,\nСтолбы, деревья, небо, лед\nИ даже иней на оконце.\nСтудент угрюм. Экзамен ждет.\n")
```

```
Светает. Радуетя солнце,  
Столбы, деревья, небо, лед  
И даже иней на оконце.  
Студент угрюм. Экзамен ждет.
```

Python поддерживает форматированный вывод строк с помощью оператора форматирования `%`. Например:

```
► print("У Миши %d яблок, а у Маши %d яблок" %(5,6))
```

```
У Миши 5 яблок, а у Маши 6 яблок
```

- Явное преобразование типов — преобразование через вызов функций типов `str()`, `int()`, `float()`.
- Неявное преобразование типов — преобразование во время исполнения кода в зависимости от контекста.

Чтобы понимать, в каком порядке выполняются операторы, нужно знать приоритеты выполнения операторов. Они перечислены в таблице ниже, где 1 — наибольший приоритет, 4 — наименьший приоритет.

Оператор	Приоритет
$x^{**}y$	1
$-x$	2
$x*y$	3
x/y	3
$x\%y$	3
$x//y$	3
$x+y$	4
$x-y$	4

Если выражение содержит несколько операторов с разными приоритетами, то сначала будут выполняться операторы с более высоким приоритетом.

Условный оператор (*if, else, elif*):

Классическая задача — проверка числа на четность. Для этого используется знак `%`, который позволяет посчитать остаток от деления, например:

```
In [33]: ► print(13%2)
```

1

```
In [34]: ► print(10%2)
```

0

Пример реализации условия в Python. В начале пишется ключевое слово `if`, затем блок кода, который должен выполняться при выполненном условии. Затем, если условие не выполняется, но есть еще одно условие, то пишется ключевое слово `elif`, после чего снова идет блок кода. Если же ни одно из условий не выполняется, то пишется ключевое слово `else` и соответствующий ему блок кода. В Python блоки кода выделяются отступом.

```
if Условие 1:
    Блок 1
elif Условие 2:
    Блок 2
else:
    Блок 3
```

Пример:

```
In [33]: ► inp = int(input('Введите целое число: '))
if inp % 2 == 0:
    print('Число четное')
else:
    print('Число нечетное')
```

Введите целое число: 11
Число нечетное

```
In [34]: ► inp = int(input('Введите целое число: '))
if inp % 2 == 0:
    print('Число четное')
else:
    print('Число нечетное')
```

Введите целое число: 4
Число четное

Структуры данных Python

В Python все структуры данных являются объектами. У них есть методы и поля. Структуры данных Python: список (list), кортеж (tuple), множество (set), словарь (dict).

Список (list).

Список – это изменяемая структура данных. Может хранить в себе элементы разных типов. Обращение к элементу списка происходит с помощью [индекса], начиная с 0. Чтобы обратиться к элементу с конца списка, то используется -1, -2 и т.д. Объявление списка происходит с помощью [] или list(). Пример:

```
❏ L1 = [1, 'Hello', 3, 'World', [1, 1, 1, 1, 1]] #элементами списка также могут быть другие списки
   L2 = [6, 7, 8, 9, 10]
```

```
❏ print(type(L2))

<class 'list'>
```

```
❏ print(L1)

[1, 'Hello', 3, 'World', [1, 1, 1, 1, 1]]
```

```
❏ print(L1[0:3])
   print(L1[-1])
   print(L1[-3:-1])

[1, 'Hello', 3]
[1, 1, 1, 1, 1]
[3, 'World']
```

```
❏ print(len(L2)) #количество элементов в списке

5
```

```
❏ L3 = L2+L2
   print(L3)

[6, 7, 8, 9, 10, 6, 7, 8, 9, 10]
```

```
❏ L2.append('Новое значение') #добавление элемента в конец списка
   L2.insert(1, 'New') #вставка элемента в определенный индекс списка
```

```
❏ print(L2)

[6, 'New', 7, 8, 9, 10, 'Новое значение']
```

```
❏ L2.remove(6) #удаления первой 6, начиная обход списка с 0 индекса
   print(L2)

['New', 7, 8, 9, 10, 'Новое значение']
```

```
» S = [4,5,8,2,1,3,4,9]
```

```
» S.sort() #сортировка списка в порядке возрастания  
print(S)
```

```
[1, 2, 3, 4, 4, 5, 8, 9]
```

```
» S.sort(reverse = True) #сортировка списка в порядке убывания  
print(S)
```

```
[9, 8, 5, 4, 4, 3, 2, 1]
```

```
» print(S.count(4)) #количество 4-рок в списке
```

```
2
```

```
» d = S.pop(0) #удаление и возвращение переменной по индексу  
print(d)  
print(S)
```

```
9
```

```
[8, 5, 4, 4, 3, 2, 1]
```

```
» L1.reverse() #перевернуть список  
print(L1)
```

```
[[1, 1, 1, 1, 1], 'World', 3, 'Hello', 1]
```

```
» print(S)
```

```
[8, 5, 4, 4, 3, 2, 1]
```

```
» del(S[0]) #удаление элемента списка
```

```
» S
```

```
]: [5, 4, 4, 3, 2, 1]
```

```
» print(max(S))  
print(min(S))
```

```
5
```

```
1
```

Можно получить список с помощью метода `split()` при вводе от пользователя, например:

```
1 a = input('Ввод: ').split()
```

```
Ввод: 1 2 3 4 5 6 7 8
```

```
1 print(a, type(a))
```

```
['1', '2', '3', '4', '5', '6', '7', '8'] <class 'list'>
```

Кортеж (tuple).

Кортеж – неизменяемая структура данных. Аналогично списку, может хранить элементы разных типов. Кортеж объявляется через () или tuple().

Кортежи работают быстрее списков за счет своей неизменяемости. Они хранятся в памяти особым образом, поэтому операции с элементами кортежа выполняются быстрее. Кортежи используются, когда есть требования к безопасности.

```
In [61]: 1 t1 = tuple('Hello')  
2 t2 = (4,6,7,1,2)  
3 t3 = 'c',
```

```
In [62]: 1 print(t1)  
2 print(t3)  
3 print(type(t3))  
  
('H', 'e', 'l', 'l', 'o')  
('c',)  
<class 'tuple'>
```

```
In [63]: 1 print(t2[1])  
2 print(t2[1:4])  
  
6  
(6, 7, 1)
```

```
In [64]: 1 print(4 in t2) #есть ли 4 в кортеже t2  
2 print(11 in t2) #есть ли 11 в кортеже t2  
  
True  
False
```

```
In [65]: 1 ts = sorted(t2) #сортировка кортежа
```

```
In [66]: 1 ts
```

```
Out[66]: [1, 2, 4, 6, 7]
```

```
In [67]: ▶ #удалить элемент из кортежа нельзя, так как он неизменяем  
#но можно удалить весь картеж  
del(ts)
```

```
In [68]: ▶ ts  
  
-----  
NameError                                Traceback (most recent call last)  
~\AppData\Local\Temp\ipykernel_2688\968267673.py in <module>  
----> 1 ts  
  
NameError: name 'ts' is not defined
```

```
In [69]: ▶ # конкатенация кортежей, аналогично спискам  
print(t1 + t2)  
  
('H', 'e', 'l', 'l', 'o', 4, 6, 7, 1, 2)
```

```
In [70]: ▶ print(t1*3)  
  
('H', 'e', 'l', 'l', 'o', 'H', 'e', 'l', 'l', 'o', 'H', 'e', 'l', 'l', 'o')
```

```
In [71]: ▶ print(t1.index('H')) #индекс элемента  
  
0
```

```
In [72]: ▶ s = ''.join(t1) #из кортежа в строку  
print(s, type(s))  
  
Hello <class 'str'>
```

```
In [73]: ▶ s = list(t1) #из кортежа в список  
print(s, type(s))  
  
['H', 'e', 'l', 'l', 'o'] <class 'list'>
```

Множество (set).

Множество – изменяемая структура данных, которая имеет требование к уникальности элементов. Множество удобно использовать для удаления повторяющихся элементов.

```
In [74]:  ► S = set('123455')
```

```
In [75]:  ► print(S)
```

```
{'2', '1', '5', '3', '4'}
```

```
In [76]:  ► S.add(10) #добавление элемента  
print(S)
```

```
{'2', '1', '5', 10, '3', '4'}
```

```
In [77]:  ► S.remove(10) #удаление элемента  
print(S)
```

```
{'2', '1', '5', '3', '4'}
```

```
In [78]:  ► S.clear() #очистка множества
```

```
In [79]:  ► print(S)
```

```
set()
```

Словарь (dict).

Словарь – ассоциативный массив, который хранит пары ключ:значение. Объявляется через {} при указании {ключ1:значение1, ключ2:значение2}. Ключ – уникальный и неизменяемый, поэтому tuple может использоваться в качестве ключа.

```
In [80]: ► T1 = {'ключ1':1, 'ключ2':2}
```

```
In [81]: ► print(T1, type(T1))  
{'ключ1': 1, 'ключ2': 2} <class 'dict'>
```

```
In [82]: ► print(len(T1))  
2
```

```
In [83]: ► print(T1.keys()) #ключи словаря  
print(T1.values()) #значения словаря  
print(T1.items()) #пары ключ:значение словаря  
  
dict_keys(['ключ1', 'ключ2'])  
dict_values([1, 2])  
dict_items([('ключ1', 1), ('ключ2', 2)])
```

```
In [84]: ► up = {'ключ3':3, 'ключ1':10}
```

```
In [85]: ► T1.update(up) #обновление словаря элементами из другого словаря
```

```
In [86]: ► # если ключ повторяется, то его значение обновляется, если ключ новый,  
# то он добавляется к словарю  
print(T1)  
  
{'ключ1': 10, 'ключ2': 2, 'ключ3': 3}
```

1 `print(T1.get('ключ1'))` # получить значение по ключу

10

1 `print(T1.pop('ключ3'))` # удаление и возврат значения по ключу

3

1 `print(T1.popitem())` # возврат и удаление последней пары в словаре
2 `print('Словарь после удаления последнего элемента: ', T1)`

('ключ2', 2)

Словарь после удаления последнего элемента: {'ключ1': 10}

1 `T2 = T1` # это неглубокая копия словаря, это ссылка на ту же ячейку памяти
2 # если изменить словарь T1, то словарь T2 тоже изменится
3 # чтобы сделать отдельный словарь, необходимо использовать `.copy()`
4 `T3 = T1.copy()`

1 `T3`

]: {'ключ1': 10}

1 `T1.update(up)` # обновление словаря элементами из другого словаря
2 `print(T2)` # словарь 2 = словарь 1
3 `print(T3)` # словарь 3 не изменился, так как это отдельный словарь

{'ключ1': 10, 'ключ3': 3}

{'ключ1': 10}

1 `import copy` # импорт библиотеки
2 `T4 = copy.deepcopy(T1)` # глубокая копия словаря

1 `T1.clear()` # очистка словаря T1
2 `print(T1)`
3 `print(T4)` # словарь T4 не изменился, он не зависит от T1

{}

{'ключ1': 10}

Циклы, функции Python и работа с библиотеками

Цикл while

Циклический оператор while используется достаточно редко, но все же бывает полезен. Определенный блок кода выполняется, пока выполняется заданное условие. Пример:

```
► #while
n = 0
while n != 10:
    print(n, end=' ')
    n+=1
```

0 1 2 3 4 5 6 7 8 9

end в выводе позволяет указать, какой знак будет стоять после вывода, по умолчанию – это \n (переход на новую строку). У циклов есть continue и break. Если в цикле используется continue, то цикл будет переходить сразу к следующей итерации, не выполняя код ниже. В случае break – происходит полное прерывание цикла. Пример:

```
► t = True
a = 0
while t:
    print(a, end = ' ')
    if a < 5:
        a+=1
        continue
    elif a>=5 and a<=10:
        a=a+2
        continue
    else:break
```

0 1 2 3 4 5 7 9 11

Цикл for

Цикл с использованием ключевого слова for – это прохождение по элементам некой последовательности. Например, необходимо вывести элементы, хранящиеся в списке:

```
▶ #for
l = [1,2,3,4,5]
for x in l:
    print(x, end = ' ')

1 2 3 4 5
```

Второй способ – это вывод элементов списка по их индексу:

```
l = [1,2,3,4,5]
for i in range(len(l)):
    print(l[i], end = ' ')

1 2 3 4 5
```

range – это генератор чисел, в данном случае он генерирует индексы от 0 до длины списка-1, то есть 0,1,2,3,4, затем вывод происходит через обращение к элементу списка по индексу. Примеры использования генератора:

```
▶ print([i for i in range(1,10,2)])

[1, 3, 5, 7, 9]
```

range(start, stop, step). start – начало, stop – конечное значение (обратите внимание, что stop не входит в интервал генерации чисел), step – шаг.

```
▶ x=range(5)
print(x, type(x))
print(list(x))

range(0, 5) <class 'range'>
[0, 1, 2, 3, 4]
```

Функции

Функция – блок кода, который выполняется при его вызове в ходе выполнения программы.

Функция в Python начинается с ключевого слова def, далее идет имя функции и (). В () указываются аргументы функции. Сначала идут аргументы, который не даны по умолчанию, затем аргументы, которые даны по умолчанию. Тело функции выделяется отступом. Пример:

```

▶ # Функции
def function(x, y=2):
    if x%y == 0:
        return ('Число нацело делится на %d'%y)
    else:
        return ('Число нацело не делится на %d'%y)

print(function(4))
print(function(4,3))

```

Число нацело делится на 2
Число нацело не делится на 3

Ключевое слово return – это результат, который возвращает функция.

Библиотека numpy

numpy это библиотека для работы с матрицами и векторами.

Для работы с библиотекой ее необходимо импортировать с помощью команды import.

```

▶ import numpy as np

```

Генерация массива в numpy:

```

▶ a = np.arange(5,10)
print(a, type(a))

```

[5 6 7 8 9] <class 'numpy.ndarray'>

Можно привести список к numpy массиву:

```

▶ x = np.array([1,2,3,4,5],dtype=float)
print(x, type(x))

```

[1. 2. 3. 4. 5.] <class 'numpy.ndarray'>

dtype позволяет указать тип хранимых элементов, в данном случае – это вещественные числа.

Двумерные (и не только) массивы:

```

▶ 1 xx = np.array([[1,1,1],[2,2,2],[3,3,3]])
  2 print(xx)

```

[[1 1 1]
[2 2 2]
[3 3 3]]

```
▶ print(xx.ndim) #кол-во размерностей  
print(xx.shape) #кол-во строк и столбцов
```

```
2  
(3, 3)
```

```
▶ #индексация  
print(xx[0])  
print(xx[1,0])
```

```
[1 1 1]  
2
```

```
▶ #операции с векторами и матрицами  
print(xx*xx) # умножение поэлементное  
print(xx@xx) # умножение матричное  
print(xx.dot(xx)) # умножение матричное  
print(xx+xx) # сложение  
print(xx-xx) # вычитание  
print(xx/xx) # деление
```

```
[[1 1 1]  
 [4 4 4]  
 [9 9 9]]  
[[ 6  6  6]  
 [12 12 12]  
 [18 18 18]]  
[[ 6  6  6]  
 [12 12 12]  
 [18 18 18]]  
[[2 2 2]  
 [4 4 4]  
 [6 6 6]]  
[[0 0 0]  
 [0 0 0]  
 [0 0 0]]  
[[1. 1. 1.]  
 [1. 1. 1.]  
 [1. 1. 1.]]
```

```
▶ #транспонирование матрицы  
print(xx.T)
```

```
[[1 2 3]  
 [1 2 3]  
 [1 2 3]]
```

`np.random.rand()` – генерация случайных значений из равномерного распределения от 0 до 1.

```
1 print(np.random.rand())  
0.4893602888250862
```

```
1 print(np.random.rand(3,2))  
[[0.76598118 0.67268707]  
 [0.61702042 0.60233133]  
 [0.9413086  0.07138761]]
```

`np.random.randn()` генерация случайных значений из нормального распределения с дисперсией, равной 1, и средним, равным 0.

Библиотека pandas

Pandas – это библиотека для обработки и анализа данных. В pandas есть две структуры объекта: DataFrame и Series. Series – это аналог одномерного массива, который имеет индексы, по сути, это ассоциативный массив, как словарь. DataFrame имеет табличную структуру, столбцы у DataFrame – это Series.

```
1 import pandas as pd  
  
1 S = pd.Series(['Мышка', 'Норушка'])  
2 print(S)  
  
0    Мышка  
1    Норушка  
dtype: object  
  
1 print(S.index) #индексы  
2 print(list(S.index)) #индексы  
3  
4 print(S.values) #список значений  
  
RangeIndex(start=0, stop=2, step=1)  
[0, 1]  
['Мышка' 'Норушка']  
  
1 print(S[0], S[1])  
  
Мышка Норушка
```

```

1 #индексы можно задать ручками
2 new_s = pd.Series(S.values, index=['Лягушка', 'Квакушка'])
3 new_s['3']=1 #добавление нового значения
4 print(new_s)

```

```

Лягушка      Мышка
Квакушка     Норушка
3              1
dtype: object

```

```

1 #групповое присваивание
2 new_s[['Лягушка', 'Квакушка']]=3,2
3 new_s.index=[1,2,3]
4 print(new_s)

```

```

1      3
2      2
3      1
dtype: object

```

```

1 print(new_s[new_s>=2]) #вывод элементов больше или равным 2

```

```

1      3
2      2
dtype: object

```

DataFrame

Загрузка данных из файла .csv в DataFrame.

```

data = pd.read_csv('Данные.csv', sep = ';', dtype=float)

```

```

data

```

```

:]:

```

	A	B	C
0	1.0	2.0	6.0
1	2.0	4.0	12.0
2	3.0	6.0	18.0
3	4.0	8.0	24.0
4	5.0	10.0	30.0
5	6.0	12.0	36.0
6	7.0	14.0	42.0
7	8.0	16.0	48.0
8	9.0	18.0	54.0

Аргумент `sep` позволяет указать разделитель. Также доступно чтение из `txt` (`read_txt`) файла и `xlsx` файла (`read_excel`).

Задать столбцы и значения можно вручную:

```
frame = pd.DataFrame({'A':range(1,11), 'B':range(100,1001,100)})
```

```
print(frame['A'], type(frame['A']))
```

```
0    1
1    2
2    3
3    4
4    5
5    6
6    7
7    8
8    9
9   10
```

```
Name: A, dtype: int64 <class 'pandas.core.series.Series'>
```

Полезным является метод `describe()`, который позволяет узнать основные статистические показатели по столбцам датафрейма.

```
print(data.columns)
print(data.index)
print(data.values)
```

```
Index(['A', 'B', 'C'], dtype='object')
```

```
RangeIndex(start=0, stop=9, step=1)
```

```
[[ 1.  2.  6.]
 [ 2.  4. 12.]
 [ 3.  6. 18.]
 [ 4.  8. 24.]
 [ 5. 10. 30.]
 [ 6. 12. 36.]
 [ 7. 14. 42.]
 [ 8. 16. 48.]
 [ 9. 18. 54.]]
```

```
print(data.describe())
```

	A	B	C
count	9.000000	9.000000	9.000000
mean	5.000000	10.000000	30.000000
std	2.738613	5.477226	16.431677
min	1.000000	2.000000	6.000000
25%	3.000000	6.000000	18.000000
50%	5.000000	10.000000	30.000000
75%	7.000000	14.000000	42.000000
max	9.000000	18.000000	54.000000

Можно создать новый столбец, используя известные данные:

```
data['D']=data['A']+data['B']+data['C']
print(data)
```

	A	B	C	D
0	1.0	2.0	6.0	9.0
1	2.0	4.0	12.0	18.0
2	3.0	6.0	18.0	27.0
3	4.0	8.0	24.0	36.0
4	5.0	10.0	30.0	45.0
5	6.0	12.0	36.0	54.0
6	7.0	14.0	42.0	63.0
7	8.0	16.0	48.0	72.0
8	9.0	18.0	54.0	81.0

Значения в DataFrame или Series можно отфильтровать:

```
1 data[(data['B']>=8) & (data['C']<48)]
```

	A	B	C	D
3	4.0	8.0	24.0	36.0
4	5.0	10.0	30.0	45.0
5	6.0	12.0	36.0	54.0
6	7.0	14.0	42.0	63.0

Обращение к датафрейму, метод `iloc` – индексация по позиции:

```
1 data.iloc[[0,1,4],[0,1]]
```

	A	B
0	1.0	2.0
1	2.0	4.0
4	5.0	10.0

Метод `loc` – индексация на основе имени:

```
1 data.loc[[0,1,4],['B','C']]
```

	B	C
0	2.0	6.0
1	4.0	12.0
4	10.0	30.0

Так же с помощью loc можно фильтровать данные, например, выведем данные, где значение признака $A > 3$ и значение признака $B < 16$:

```
1 data.loc[(data.A > 3) & (data.B < 16)]
```

	A	B	C	D
3	4.0	8.0	24.0	36.0
4	5.0	10.0	30.0	45.0
5	6.0	12.0	36.0	54.0
6	7.0	14.0	42.0	63.0

Метод `apply()` позволяет применить к серии или датафрейму некоторую функцию, например, создание нового признака, если значение в столбце A меньше 5 или больше:

```
1 def function(x):
2     if x['A'] < 5:
3         x['new'] = x['A']
4     else:
5         x['new'] = x['A'] ** 2
6     return x
7
8 a = data.apply(function, axis = 1)
9 print(a)
```

	A	B	C	D	new
0	1.0	2.0	6.0	9.0	1.0
1	2.0	4.0	12.0	18.0	2.0
2	3.0	6.0	18.0	27.0	3.0
3	4.0	8.0	24.0	36.0	4.0
4	5.0	10.0	30.0	45.0	25.0
5	6.0	12.0	36.0	54.0	36.0
6	7.0	14.0	42.0	63.0	49.0
7	8.0	16.0	48.0	72.0	64.0
8	9.0	18.0	54.0	81.0	81.0

$axis = 1$ – это проход по строкам, $axis = 0$ – это проход по столбцам, например:

```

1 def f(x):
2     mean = x.mean()
3     print(x.name, mean)
4     return x - mean

```

```

1 print(data.apply(f,axis = 0))

```

```

A 5.0
B 10.0
C 30.0
D 45.0

```

	A	B	C	D
0	-4.0	-8.0	-24.0	-36.0
1	-3.0	-6.0	-18.0	-27.0
2	-2.0	-4.0	-12.0	-18.0
3	-1.0	-2.0	-6.0	-9.0
4	0.0	0.0	0.0	0.0
5	1.0	2.0	6.0	9.0
6	2.0	4.0	12.0	18.0
7	3.0	6.0	18.0	27.0
8	4.0	8.0	24.0	36.0

Библиотека *scikit-learn*.

scikit-learn – это библиотека для эффективного предиктивного анализа данных. Она содержит в себе различные алгоритмы машинного обучения, инструменты для предобработки данных, готовые наборы данных для исследований и тд.

Готовые датасеты можно взять в модуле dataset. У каждого датасета есть функция для его загрузки.

Рассмотрим пример загрузки датасета Ames housing:

```

1 from sklearn.datasets import fetch_openml

```

```

1 df = fetch_openml(name ="house_prices", as_frame=True)
2 print(type(df))

```

```
<class 'sklearn.utils._bunch.Bunch'>
```

В результате df – это объект Bunch, который представляет собой аналог словаря в библиотеке Scikit-Learn. Соответственно, у него есть ключи.

```

1 df.keys()

```

```
dict_keys(['data', 'target', 'frame', 'categories', 'feature_names', 'target_names', 'DESCR', 'details', 'url'])
```

data – это таблица признаков. target – это значение целевой переменной. Этот набор данных используется для прогнозирования стоимости дома.

1df.data

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	ScreenPorch	PoolArea	PoolQC	Fence	
0	1.0	60.0	RL	65.0	8450.0	Pave	None	Reg	Lvl	AllPub	...	0.0	0.0	None	None	
1	2.0	20.0	RL	80.0	9600.0	Pave	None	Reg	Lvl	AllPub	...	0.0	0.0	None	None	
2	3.0	60.0	RL	68.0	11250.0	Pave	None	IR1	Lvl	AllPub	...	0.0	0.0	None	None	
3	4.0	70.0	RL	60.0	9550.0	Pave	None	IR1	Lvl	AllPub	...	0.0	0.0	None	None	
4	5.0	60.0	RL	84.0	14260.0	Pave	None	IR1	Lvl	AllPub	...	0.0	0.0	None	None	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
1455	1456.0	60.0	RL	62.0	7917.0	Pave	None	Reg	Lvl	AllPub	...	0.0	0.0	None	None	
1456	1457.0	20.0	RL	85.0	13175.0	Pave	None	Reg	Lvl	AllPub	...	0.0	0.0	None	MnPrv	
1457	1458.0	70.0	RL	66.0	9042.0	Pave	None	Reg	Lvl	AllPub	...	0.0	0.0	None	GdPrv	
1458	1459.0	20.0	RL	68.0	9717.0	Pave	None	Reg	Lvl	AllPub	...	0.0	0.0	None	None	
1459	1460.0	20.0	RL	75.0	9937.0	Pave	None	Reg	Lvl	AllPub	...	0.0	0.0	None	None	

1460 rows × 80 columns

Подробнее данную библиотеку рассмотрим в следующих практических работах.

Практическая работа

1. Установить Python, если это не было сделано ранее.
2. Написать программу, которая вычисляет площадь фигуры, параметры которой подаются на вход. Фигуры, которые подаются на вход: треугольник, прямоугольник, круг. Результатом работы является словарь, где ключ – это название фигуры, а значение – это площадь.
3. Написать программу, которая на вход получает два числа и операцию, которую к ним нужно применить. Должны быть реализованы следующие операции: +, -, /, //, abs – модуль, pow или ** – возведение в степень.
4. Напишите программу, которая считывает с консоли числа (по одному в строке) до тех пор, пока сумма введённых чисел не будет равна 0 и после этого выводит сумму квадратов всех считанных чисел.
5. Напишите программу, которая выводит последовательность чисел, длиною N, где каждое число повторяется столько раз, чему оно равно. На вход программе передаётся неотрицательное целое число N. Например, если N = 7, то программа должна вывести 1 2 2 3 3 3 4. Вывод элементов списка через пробел – `print(*list)`.
6. Даны два списка:
`A = [1, 2, 3, 4, 2, 1, 3, 4, 5, 6, 5, 4, 3, 2]`
`B = ['a', 'b', 'c', 'c', 'c', 'b', 'a', 'c', 'a', 'a', 'b', 'c', 'b', 'a']`
Создать словарь, в котором ключи – это содержимое списка B, а значения для ключей словаря – это сумма всех элементов списка A в соответствии с буквой, содержащийся на той же позиции в списке B.
Пример результата программы: `{'a': 10, 'b': 15, 'c': 6}`.
Скачать и загрузить данные о стоимости домов в калифорнии, используя библиотеку `sklearn`.

```
➤ 1 from sklearn.datasets import fetch_california_housing  
   2 data = fetch_california_housing(as_frame=True)
```

8. Использовать метод `info()`.

9. Узнать, есть ли пропущенные значения, используя `isna().sum()`.
10. Вывести записи, где средний возраст домов в районе более 50 лет и население более 2500 человек, используя метод `loc()`.
11. Узнать максимальное и минимальное значения медианной стоимости дома.
12. Используя метод `apply()`, вывести на экран название признака и его среднее значение.
13. Составить отчет о проделанной работе. В отчете должен быть представлен код и результаты его выполнения с выводами.

1.*

Дан текст на английском языке. Необходимо закодировать его с помощью азбуки Морзе, где каждой букве соответствует последовательность точек и тире. Например, буква «g» превратится в строку «--.». В переменной `morze` для удобства хранится словарь соответствия латинских букв коду Морзе.

```
morze = {'a': '-.-', 'b': '-...', 'c': '-.-.', 'd': '-..',  
        'e': '.', 'f': '..-.', 'g': '--.', 'h': '....',  
        'i': '..', 'j': '---.', 'k': '-.-', 'l': '-.-.',  
        'm': '--', 'n': '-.', 'o': '---', 'p': '---.',  
        'q': '--.-', 'r': '-.-.', 's': '...', 't': '-',  
        'u': '..-', 'v': '...-', 'w': '---', 'x': '-.-.-',  
        'y': '-.-.-', 'z': '---..'}

```

На входе: В одной строке вам дан текст, который состоит из латинских букв и пробелов.

На выходе:

Выведите каждое слово исходного текста, закодированное азбукой Морзе. Количество строк в ответе должно совпадать с количеством слов

в исходном тексте. Между закодированными буквами ставится ровно один пробел. Например, слово «Help» превратится в «.... . -.. ---». Строчные и заглавные буквы кодируются одинаково.

Например:

Ignition sequence start

Перевод

.. --. -.. -.. --- -.
.... -.- -.. -.. -.-..
... -.- -.- -

2.*

В некотором городе открывается новая служба по доставке электронных писем. Необходимо наладить систему регистрации новых пользователей.

Регистрация должна работать следующим образом: если новый пользователь хочет зарегистрироваться на сайте, то он должен послать системе запрос name со своим именем. Система должна определить, существует ли уже такое имя в базе данных. Если такого имени не существует, то оно заносится в базу данных системы и пользователю возвращается ответ "ОК", подтверждающий успешную регистрацию. А если пользователь с таким именем уже существует, то система должна сформировать новое имя и выдать его пользователю в качестве подсказки, при этом сама подсказка также добавляется в базу данных. Новое имя формируется следующим образом: к name последовательно приписываются числа, начиная с 1 (name1, name2 и так далее), и среди них находят такое наименьшее i , что name i еще не содержится в системе.

Входные данные

В первой строке входных данных задано число n ($1 \leq n \leq 100000$). Следующие n строк содержат запросы к системе. Каждый запрос

представляет собой непустую строку длиной не более 32 символов, состоящую только из строчных букв латинского алфавита.

Выходные данные

В выходных данных должно содержаться n строк – ответы системы на запросы: "OK" в случае успешной регистрации, или подсказка с новым именем, если запрашиваемое уже занято.

Данные для проверки:

Вход:	Выход:
3 b b b	OK b1 b2
10 bhnqaptmp bhnqaptmp bhnqaptmp bhnqaptmp bhnqaptmp bhnqaptmp bhnqaptmp bhnqaptmp bhnqaptmp bhnqaptmp	OK bhnqaptmp1 bhnqaptmp2 bhnqaptmp3 bhnqaptmp4 bhnqaptmp5 bhnqaptmp6 bhnqaptmp7 bhnqaptmp8 bhnqaptmp9
10 fpqhfuqlddravpjttarh fpqhfuqlddravpjttarh fpqhfuqlddravpjttarh fpqhfuqlddravpjttarh fpqhfuqlddravpjttarh fpqhfuqlddravpjttarh jmvplnrmba fpqhfuqlddravpjttarh jmvplnrmba fpqhfuqlddravpjttarh	OK fpqhfuqlddravpjttarh1 fpqhfuqlddravpjttarh2 fpqhfuqlddravpjttarh3 fpqhfuqlddravpjttarh4 fpqhfuqlddravpjttarh5 OK fpqhfuqlddravpjttarh6 jmvplnrmba1 fpqhfuqlddravpjttarh7

3.*

Необходимо создать программу обработки запросов пользователей к файловой системе компьютера. Над каждым файлом можно производить следующие действия: запись – w ("write"), чтение – r ("read"), запуск – x ("execute").

Входные данные

На вход программе подаются следующие параметры: число n – количество файлов в файловой системе. В следующих n строках содержится информация с именами файлов и допустимыми действиями (w, x, r), разделенных пробелами. Далее идет число m – количество запросов к файлам вида «операция файл» (обозначение операции: "write", "read", "execute").

Выходные данные

Для каждого допустимого запроса программа должна возвращать ОК, для недопустимого – Access denied.

Данные для проверки:

Вход:	Выход:
3 python.exe x book.txt r w notebook.exe r w x	
5 read python.exe read book.txt write notebook.exe execute notebook.exe write book.txt	Access denied OK OK OK OK
3 root.html r w x main.py x login.txt w r	
4 read root.html write main.py execute main.py execute login.txt	OK Access denied OK Access denied

2 1.txt 2.txt 2 write 1.txt execute 2.txt	Access denied Access denied
--	--------------------------------